

EQUAZIONI DI RICORRENZA - ESERCIZI

PIETRO DI LENA

DIPARTIMENTO DI INFORMATICA – SCIENZA E INGEGNERIA
UNIVERSITÀ DI BOLOGNA

ALGORITMI E STRUTTURE DI DATI
ANNO ACCADEMICO 2024/2025



ESERCIZIO 1

- Risolvere la seguente equazione di ricorrenza

$$T(n) = \begin{cases} d & n \leq 1 \\ T(n/4) + c & n > 1 \end{cases}$$

con

- Master Theorem
- Metodo dell'iterazione

ESERCIZIO 1 - SOLUZIONE

■ Master Theorem

$$T(n) = \begin{cases} d & n \leq 1 \\ T(n/4) + c & n > 1 \end{cases}$$

Abbiamo $a = 1$, $b = 4$, $\alpha = \log_b a = 0$, $\beta = 0$

$$\alpha = \beta \Rightarrow T(n) = \Theta(n^0 \log n) = \Theta(\log n)$$

ESERCIZIO 1 - SOLUZIONE

■ Metodo dell'iterazione

$$T(n) = \begin{cases} d & n \leq 1 \\ T(n/4) + c & n > 1 \end{cases}$$

Abbiamo

$$\begin{aligned} T(n) &= T(n/4) + c \\ &= T(n/4^2) + c + c \\ &= T(n/4^3) + c + c + c \\ &\dots \\ &= T(n/4^i) + c \cdot i \end{aligned}$$

La ricorsione termina quando $n/4^i = 1 \Rightarrow i = \log_4 n$. Quindi

$$T(n) = T(1) + c \log_4 n = d + c \log_4 n = \Theta(\log n)$$

ESERCIZIO 2

- Risolvere la seguente equazione di ricorrenza

$$T(n) = \begin{cases} d & n \leq 1 \\ 2T(n/4) + c & n > 1 \end{cases}$$

con

- Master Theorem
- Metodo dell'iterazione

ESERCIZIO 2 - SOLUZIONE

■ Master Theorem

$$T(n) = \begin{cases} d & n \leq 1 \\ 2T(n/4) + c & n > 1 \end{cases}$$

Abbiamo $a = 1$, $b = 4$, $\alpha = \log_4 2 = 1/2$, $\beta = 0$

$$\alpha > \beta \Rightarrow T(n) = \Theta(n^\alpha) = \Theta(\sqrt{n})$$

ESERCIZIO 2 - SOLUZIONE

■ Metodo dell'iterazione

$$T(n) = \begin{cases} d & n \leq 1 \\ 2T(n/4) + c & n > 1 \end{cases}$$

Abbiamo

$$\begin{aligned} T(n) &= 2T(n/4) + c \\ &= 2^2 T(n/4^2) + 2c + c \\ &= 2^3 T(n/4^3) + 2^2 c + 2c + c \\ &\dots \end{aligned}$$

$$= 2^i T(n/4^i) + \sum_{k=0}^{i-1} 2^k c$$

La ricorsione termina quando $n/4^i = 1 \Rightarrow i = \log_4 n$. Quindi

$$T(n) = 2^{\log_4 n} T(1) + c \frac{2^{\log_4 n} - 1}{2 - 1} = \sqrt{n}d + c(\sqrt{n} - 1) = \Theta(\sqrt{n})$$

ESERCIZIO 3

- Risolvere la seguente equazione di ricorrenza

$$T(n) = \begin{cases} d & n \leq 1 \\ 4T(n/4) + c & n > 1 \end{cases}$$

con

- Master Theorem
- Metodo dell'iterazione

ESERCIZIO 3 - SOLUZIONE

■ Master Theorem

$$T(n) = \begin{cases} d & n \leq 1 \\ 4T(n/4) + c & n > 1 \end{cases}$$

Abbiamo $a = 1$, $b = 4$, $\alpha = \log_4 4 = 1$, $\beta = 0$

$$\alpha > \beta \Rightarrow T(n) = \Theta(n^\alpha) = \Theta(n)$$

ESERCIZIO 3 - SOLUZIONE

■ Metodo dell'iterazione

$$T(n) = \begin{cases} d & n \leq 1 \\ 4T(n/4) + c & n > 1 \end{cases}$$

Abbiamo

$$\begin{aligned} T(n) &= 4T(n/4) + c \\ &= 4^2 T(n/4^2) + 4c + c \\ &= 4^3 T(n/4^3) + 4^2 c + 4c + c \end{aligned}$$

..

$$= 4^i T(n/4^i) + \sum_{k=0}^{i-1} 4^k c$$

La ricorsione termina quando $n/4^i = 1 \Rightarrow i = \log_4 n$. Quindi

$$T(n) = 4^{\log_4 n} T(1) + c \frac{4^{\log_4 n} - 1}{4 - 1} = nd + c \frac{n - 1}{3} = \Theta(n)$$

ESERCIZIO 4

- Risolvere la seguente equazione di ricorrenza

$$T(n) = \begin{cases} d & n \leq 1 \\ 8T(n/4) + c & n > 1 \end{cases}$$

con

- Master Theorem
- Metodo dell'iterazione

ESERCIZIO 4 - SOLUZIONE

■ Master Theorem

$$T(n) = \begin{cases} d & n \leq 1 \\ 8T(n/4) + c & n > 1 \end{cases}$$

Abbiamo $a = 1, b = 4, \alpha = \log_4 8 = 3/2, \beta = 0$

$$\alpha > \beta \Rightarrow T(n) = \Theta(n^{3/2}) = \Theta(\sqrt{n^3})$$

ESERCIZIO 4 - SOLUZIONE

■ Metodo dell'iterazione

$$T(n) = \begin{cases} d & n \leq 1 \\ 8T(n/4) + c & n > 1 \end{cases}$$

Abbiamo

$$\begin{aligned} T(n) &= 8T(n/4) + c \\ &= 8^2 T(n/4^2) + 8c + c \\ &= 8^3 T(n/4^3) + 8^2 c + 8c + c \\ &\dots \\ &= 8^i T(n/4^i) + \sum_{k=0}^{i-1} 8^k c \end{aligned}$$

La ricorsione termina quando $n/4^i = 1 \Rightarrow i = \log_4 n$. Quindi

$$T(n) = 8^{\log_4 n} T(1) + c \frac{8^{\log_4 n} - 1}{8 - 1} = \sqrt{n^3} d + c \frac{\sqrt{n^3} - 1}{7} = \Theta(\sqrt{n^3})$$

Esercizio 5

Analizzare il costo computazionale $T(n)$ dell'algoritmo FIBMATPOW per calcolare l' n -esima potenza della matrice di Fibonacci con la tecnica *exponentiation by squaring*

```
1: function FIBMATPOW(INT  $n$ )  $\rightarrow$  FIBMAT
2:    $A = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$ 
3:   if  $n > 1$  then
4:      $M = \text{FIBMATPOW}(n/2)$ 
5:      $A = M \times M$ 
6:     if  $n \bmod 2 \neq 0$  then
7:        $A = A \times \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$ 
8:   return  $A$ 
```

Esercizio 5 - Soluzione

- Il costo di FIBMATPOW può essere descritto dalla seguente equazione di ricorrenza

$$T(n) = \begin{cases} 1 & n \leq 1 \\ T(n/2) + 1 & n > 1 \end{cases}$$

Secondo il Master Theorem

$$\alpha = \log_2 1 = 0 = \beta \Rightarrow T(n) = \Theta(n^0 \log n) = \Theta(\log n)$$

Esercizio 6

- Analizzare il costo computazionale $T(n)$ dell'algoritmo FIBMATPOW per calcolare l' n -esima potenza della matrice di Fibonacci con la tecnica *exponentiation by cubing*
- E' più efficiente di quella implementata con *exponentiation by squaring*?

```
1: function FIBMATPOW(INT  $n$ )  $\rightarrow$  FIBMAT
2:    $A = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}, A^2 = \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix}$ 
3:   if  $n \leq 1$  then
4:     return  $A$ 
5:   else if  $n == 2$  then
6:     return  $A^2$ 
7:   else
8:      $M = \text{FIBMATPOW}(n/3)$ 
9:     if  $n \bmod 3 == 0$  then  $\triangleright n$  multiplo di 3
10:      return  $M \times M \times M$ 
11:     else if  $n \bmod 3 == 1$  then  $\triangleright n = 3k + 1$  per qualche  $k \geq 1$ 
12:      return  $M \times M \times M \times A$ 
13:     else  $\triangleright n = 3k + 2$  per qualche  $k \geq 1$ 
14:      return  $M \times M \times M \times A^2$ 
```


Esercizio 6 - Soluzione

- Il costo di FIBMATPOW può essere descritto dalla seguente equazione di ricorrenza

$$T(n) = \begin{cases} 1 & n \leq 2 \\ T(n/3) + 1 & n > 2 \end{cases}$$

Secondo il Master Theorem

$$\alpha = \log_3 1 = 0 = \beta \Rightarrow T(n) = \Theta(n^0 \log n) = \Theta(\log n)$$

- La tecnica exponentiation by cubing non è asintoticamente più efficiente di exponentiation by squaring e comporta un algoritmo più complesso dal punto di vista implementativo

Esercizio 7

Analizzare il costo computazionale $T(n)$ dell'algoritmo MYSTERY1 in funzione del valore in input n

```
1: function MYSTERY1(INT  $n$ )  $\rightarrow$  INT
2:   if  $n \leq 1$  then
3:     return 123
4:   else
5:      $k = \text{MYSTERY2}(n/2) + \text{MYSTERY1}(n/3)$ 
6:     return  $k + \text{MYSTERY1}(n/3)$ 

7: function MYSTERY2(INT  $n$ )  $\rightarrow$  INT
8:   if  $n == 0$  then
9:     return 321
10:  else
11:    return  $2 * \text{MYSTERY2}(n/4) - \text{MYSTERY2}(n/4)$ 
```

Esercizio 7 - Soluzione

- Il costo di MYSTERY2 può essere descritto dalla seguente equazione di ricorrenza

$$T'(n) = \begin{cases} 1 & n = 0 \\ 2T'(n/4) + 1 & n > 0 \end{cases}$$

Secondo il Master Theorem

$$\alpha = \log_4 2 = 1/2 > 0 = \beta \Rightarrow T'(n) = \Theta(\sqrt{n})$$

- Il costo di MYSTERY1 può essere descritto dalla seguente equazione di ricorrenza

$$T(n) = \begin{cases} 1 & n \leq 1 \\ 2T(n/3) + n^{1/2} & n > 1 \end{cases}$$

Secondo il Master Theorem

$$\alpha = \log_3 2 \approx 0.63 > 0.5 = \beta \Rightarrow T(n) = \Theta(n^{\log_3 2})$$

Esercizio 8

Analizzare il costo computazionale $T(n)$ dell'algoritmo MYSTERY1 in funzione del valore in input n

```
1: function MYSTERY1(INT  $n$ )  $\rightarrow$  INT
2:   if  $n \leq 1$  then
3:     return 32
4:   else
5:     return MYSTERY2( $n/2$ )+MYSTERY1( $n/2$ )

6: function MYSTERY2(INT  $n$ )  $\rightarrow$  INT
7:   if  $n == 1$  then
8:     return 2
9:   else
10:    return 2*MYSTERY2( $n - 1$ )
```

Esercizio 8 - Soluzione 1/2

- Il costo di MYSTERY2 può essere descritto dalla seguente equazione di ricorrenza

$$T'(n) = \begin{cases} 1 & n = 1 \\ T'(n-1) + 1 & n > 1 \end{cases}$$

Possiamo risolvere tale equazione di ricorrenza utilizzando il metodo iterativo

$$\begin{aligned} T'(n) &= T'(n-1) + 1 \\ &= T'(n-2) + 1 + 1 \\ &= T'(n-3) + 1 + 1 + 1 \\ &\dots \\ &= T'(n-i) + i \end{aligned}$$

La ricorsione termina quando $n - i = 1 \Rightarrow i = n - 1$. Concludiamo quindi che

$$T'(n) = 1 + n - 1 = \Theta(n)$$

Esercizio 8 - Soluzione 2/2

- Il costo di MYSTERY1 può essere descritto dalla seguente equazione di ricorrenza

$$T(n) = \begin{cases} 1 & n \leq 1 \\ T(n/2) + n & n > 1 \end{cases}$$

Secondi il Master Theorem $\alpha = \log_2 1 = 0 < 1 = \beta \Rightarrow T(n) = \Theta(n)$

Esercizio 9

Analizzare il costo computazionale $T(n)$ dell'algoritmo MYSTERY1 in funzione del valore in input n

```
1: function MYSTERY1(INT  $n$ )  $\rightarrow$  INT
2:    $i = 1$ 
3:    $j = 0$ 
4:   while  $i \leq n$  do
5:      $i = i * 2$ 
6:      $j = j + \text{MYSTERY2}(n/2) + \text{MYSTERY2}(n/2)$ 
7:   return  $j$ 

8: function MYSTERY2(INT  $n$ )  $\rightarrow$  INT
9:   if  $n \leq 1$  then
10:    return 1
11:  else
12:     $j = 1$ 
13:    while  $j \leq n$  do
14:       $j = j + 1$ 
15:    return  $j + \text{MYSTERY2}(n/3) + \text{MYSTERY2}(n/3)$ 
```

Esercizio 9 - Soluzione

- Il costo di MYSTERY2 può essere descritto dalla seguente equazione di ricorrenza (il ciclo while viene eseguito n volte)

$$T'(n) = \begin{cases} 1 & n \leq 1 \\ 2T'(n/3) + n & n > 1 \end{cases}$$

Per il Master Theorem $\alpha = \log_3 2 \approx 0.63 < 1 = \beta \Rightarrow T'(n) = \Theta(n)$

- La funzione MYSTERY1 richiama iterativamente MYSTERY2 due volte. Ogni chiamata costa $\Theta(n/2) + \Theta(n/2) = \Theta(n/2)$. Per completare l'analisi dobbiamo calcolare quante volte viene eseguito il ciclo while (right 4-6) in MYSTERY1. La variabile i (inizialmente 1) viene raddoppiata ad ogni iterazione e il ciclo termina quando $i > n$

iterazione	0	1	2	$\dots$	k
valore di i	1	2	4	$\dots$	2^k

Quindi, il ciclo while termina all'iterazione k tale per cui

$$2^k > n \Rightarrow \log_2 2^k > \log_2 n \Rightarrow k > \log_2 n$$

In altri termini, il ciclo while viene eseguito $\Theta(\log n)$ volte. Quindi

$$T(n) = \Theta(\log n) \cdot (\Theta(n/2) + \Theta(n/2)) = \Theta(n \log n)$$

Esercizio 10

Analizzare il costo computazionale $T(n)$ dell'algoritmo MYSTERY1 in funzione del valore in input n

```
1: function MYSTERY1(INT  $n$ )  $\rightarrow$  INT
2:   if  $n \leq 1$  then
3:     return 1
4:   else
5:      $i = 1$ 
6:      $j = 0$ 
7:     while  $i \leq n$  do
8:        $i = i + 2$ 
9:        $j = j + \text{MYSTERY2}(n/2)$ 
10:    return  $j + \text{MYSTERY1}(n/4) + \text{MYSTERY1}(n/4) + \text{MYSTERY1}(n/4)$ 

11: function MYSTERY2(INT  $n$ )  $\rightarrow$  INT
12:   if  $n \leq 1$  then
13:     return 1
14:   else
15:      $j = 1$ 
16:     while  $j \leq n$  do
17:        $j = j + 1$ 
18:    return  $j + \text{MYSTERY2}(n/3) + \text{MYSTERY2}(n/3)$ 
```

Esercizio 10 - Soluzione

- Il costo di MYSTERY2 può essere descritto dalla seguente equazione di ricorrenza (il ciclo while viene eseguito n volte)

$$T'(n) = \begin{cases} 1 & n \leq 1 \\ 2T'(n/3) + n & n > 0 \end{cases}$$

Per il Master Theorem $\alpha = \log_3 2 \approx 0.63 < 1 = \beta \Rightarrow T'(n) = \Theta(n)$

- Notiamo che il ciclo while (righe 7-9) viene eseguito $n/2$ volte e ad ogni iterazione richiama la funzione MYSTERY2 con input $n/2$. Quindi il ciclo while costa complessivamente $n/2 \cdot \Theta(n/2) = \Theta(n^2)$. Il costo di MYSTERY1 può quindi essere descritto dalla seguente equazione di ricorrenza

$$T(n) = \begin{cases} 1 & n \leq 1 \\ 3T(n/4) + n^2 & n > 1 \end{cases}$$

Per il Master Theorem $\alpha = \log_4 3 \approx 0.79 < 2 = \beta \Rightarrow T(n) = \Theta(n^2)$